



SOFTWARE DEV

MERN STACK DEVELOPMENT REPORT

*Week 4 : Asynchronous JavaScript, APIs, and Modular
Architecture*

Muneeb Ur Rehman

MERN STACK INTERN

<https://github.com/955muneeb/Glaxit-internship/tree/main/week-4>

Asynchronous JavaScript, APIs, and Modular Architecture (Week 4)

Name: Muneeb Ur Rehman

Position: MERN Stack Intern

1. Abstract

Modern web applications rely heavily on asynchronous programming to deliver responsive and efficient user experiences. This laboratory focused on implementing asynchronous JavaScript concepts, event-driven programming, HTTP communication using the Fetch API, robust error-handling techniques, and modular application architecture using ES6 modules.

Throughout this lab, several practical applications were developed to demonstrate these concepts. The first activity involved designing an accurate digital stopwatch using JavaScript timing functions while preventing execution conflicts caused by repeated user interactions. The second activity explored asynchronous data retrieval from remote APIs using both the Fetch API and the async/await programming model, alongside comprehensive exception handling techniques. The final activity involved designing a modular enterprise dashboard by separating application responsibilities into dedicated service, storage, and user interface modules.

These exercises provided practical experience in writing maintainable, scalable, and production-ready JavaScript applications while following modern software engineering principles.

2. Objectives

The primary objectives of this laboratory were:

- Develop an execution-safe digital stopwatch using JavaScript timing functions.
 - Understand asynchronous programming using Promises and async/await.
 - Retrieve remote data using the Fetch API.
 - Implement robust error handling using try, catch, and finally.
 - Design reusable JavaScript modules using the ES6 import/export system.
 - Apply separation of concerns by organizing application logic into independent components.
-

3. Methodology

3.1 Day 1 – Advanced Stopwatch Implementation

The first task focused on implementing an accurate digital stopwatch capable of measuring elapsed time with high precision.

Unlike traditional stopwatch implementations that rely solely on incrementing counters within a timer, this implementation calculates elapsed time using the system clock. The elapsed duration is continuously computed as:

$$\text{Elapsed Time} = \text{Date.now}() - \text{StartTime}$$

Using the system timestamp significantly reduces timing drift that may occur due to browser scheduling delays.

To prevent multiple timers from running simultaneously, an execution guard (isRunning) was introduced. This boolean flag blocks repeated clicks on the Start button, eliminating race conditions that could otherwise accelerate the stopwatch unexpectedly.

The measured time is formatted into the following structure before being displayed:

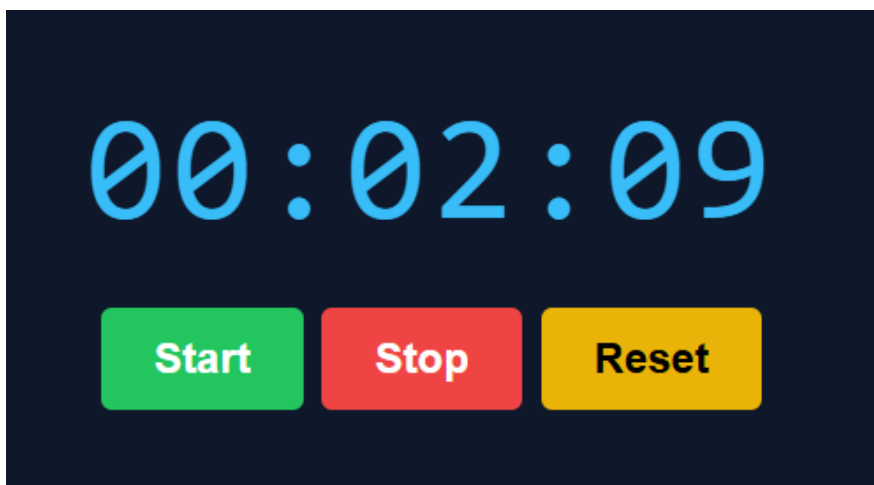
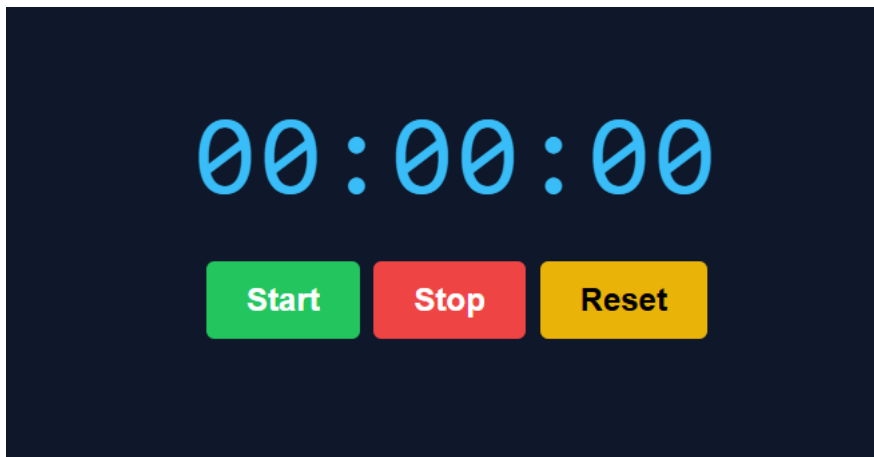
MM : SS : CS

where:

- MM = Minutes
- SS = Seconds
- CS = Centiseconds

Key Features

- High-precision stopwatch
- Race-condition prevention
- Millisecond timestamp calculation
- Zero-padded time formatting
- Start, Stop, and Reset controls



3.2 Days 2–3 – Asynchronous Data Fetching and Error Handling

The second phase of the laboratory introduced asynchronous programming concepts through the Fetch API.

Modern web applications constantly communicate with remote servers. Since these operations occur asynchronously, applications must gracefully handle network failures without crashing.

The Fetch API was used to retrieve user information from an external REST API.

The workflow consisted of the following steps:

1. Display a loading indicator.
2. Send an asynchronous HTTP request.
3. Wait for the server response.
4. Parse the JSON payload.
5. Display retrieved user information.
6. Hide the loading indicator.

The implementation utilized the `async/await` syntax to improve code readability compared to traditional Promise chaining.

HTTP Response Validation

One important observation during this exercise is that the Fetch API does **not** automatically throw exceptions for HTTP errors such as **404** or **500**.

Therefore, the response status was manually verified using:

```
if (!response.ok)

    throw new Error(`HTTP Error: ${response.status}`);
```

This ensures invalid responses are correctly handled within the exception block.

Exception Handling

All asynchronous operations were enclosed inside a `try...catch...finally` block.

- **try** executes the network request.
- **catch** handles server or network failures.
- **finally** guarantees cleanup operations regardless of success or failure.

If an invalid endpoint is requested, the application displays a user-friendly error banner instead of terminating unexpectedly.

Key Features

- Fetch API integration
- Async/Await implementation
- JSON parsing

- HTTP status validation
- Graceful error handling
- Loading indicator
- Fallback user interface

User Directory Grid

Fetch Users

Test Broken URL Error

User Directory Grid

Fetch Users

Test Broken URL Error

Leanne Graham

@Bret
Sincere@april.biz

Ervin Howell

@Antonette
Shanna@melissa.tv

Chelsey Dietrich

@Kamren
Lucio_Hettinger@annie.ca

Mrs. Dennis Schulist

@Leopoldo_Corkery
Karley_Dach@jasper.info

Glenna Reichert

@Delphine
Chaim_McDermott@dana.io

Clementina DuBuque

@Moriah.Stanton
Rey.Padberg@karina.biz

Fetch Users

Test Broken URL Error

Operation Failed: HTTP System Error: Status 404. Please verify the upstream service.

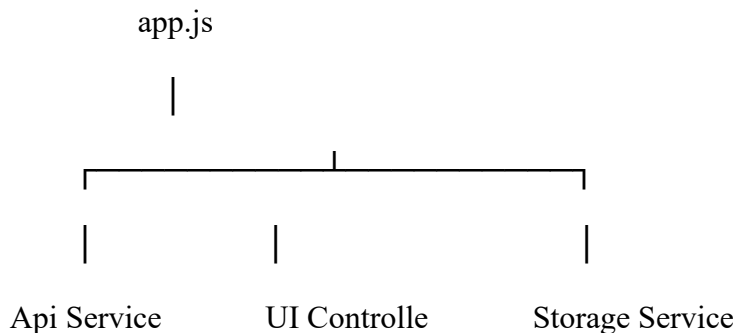
3.3 Day 5 – Modular Enterprise Dashboard

The final laboratory exercise focused on designing a modular JavaScript application using ES6 Modules.

Instead of placing all application logic inside a single JavaScript file, the project was divided into specialized modules following the **Separation of Concerns (SoC)** principle.

This approach significantly improves code maintainability, scalability, readability, and reusability.

Application Architecture



Module Responsibilities

`api.service.js`

Responsible for:

- Sending HTTP requests
 - Receiving remote data
 - Parsing JSON responses
 - Returning processed information
-

`storage.service.js`

Responsible for:

- Browser storage management
 - LocalStorage interaction
 - Data serialization
 - Data persistence
-

`ui.controller.js`

Responsible for:

- Rendering HTML components
- Updating the DOM

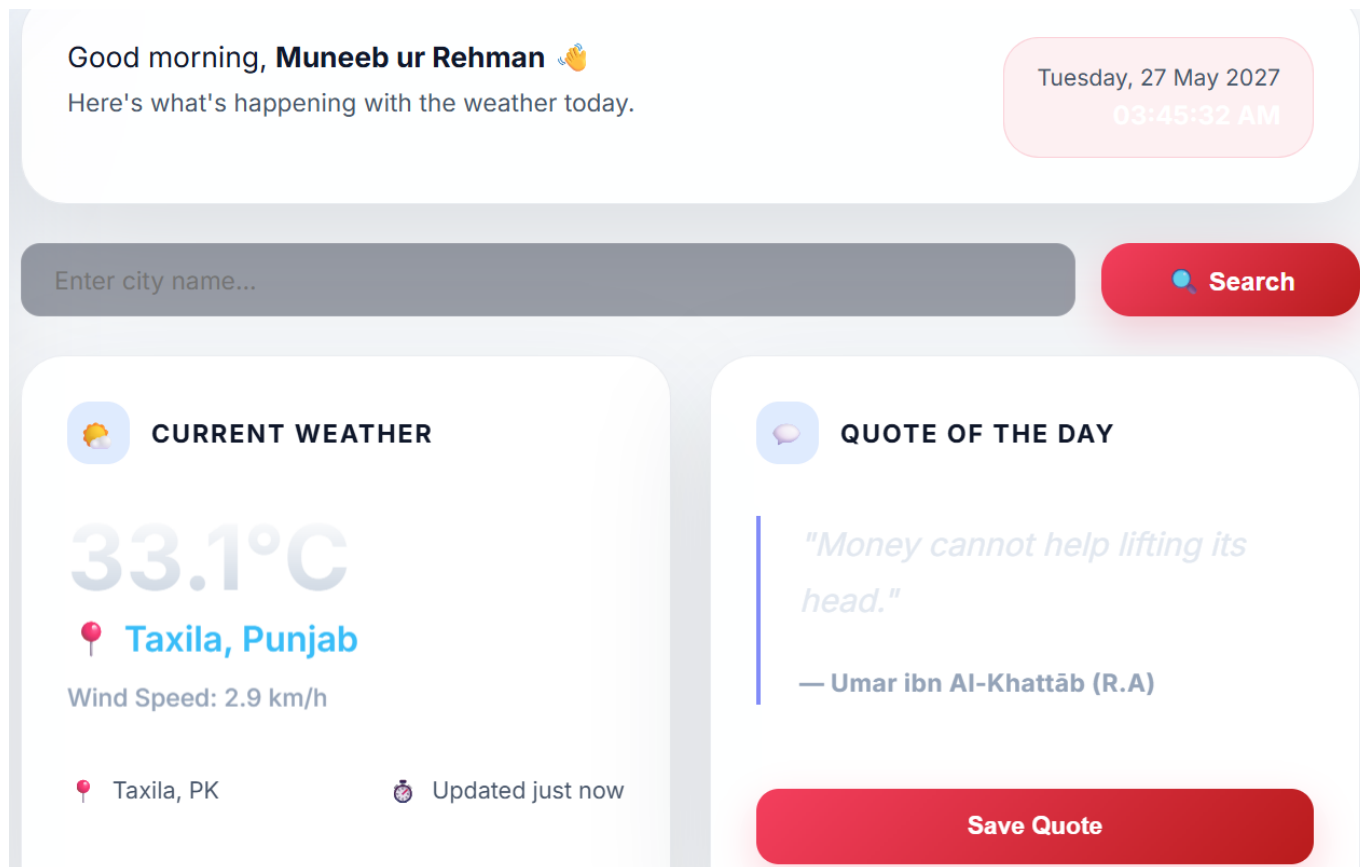
- Displaying weather information
- Rendering motivational quotes

app.js

Acts as the central controller of the application.

Its responsibilities include:

- Initializing the application
- Calling service modules
- Coordinating UI updates
- Handling startup exceptions





YOUR FAVORITE QUOTES

"Efforts And Courage Are Not Enough Without Purpose And Direction."

— John F. Kennedy

"Beautiful people are not always good, but good people are always beautiful."

— Ali ibn Abi Talib (R.A)

"Money cannot help lifting its head."

— Umar ibn Al-Khattāb (R.A)



Dashboard



Dashboard



Weather



Quotes



Favorites